CAPSTONE 24

AI Tutor for Communication Protocols

(I2C • SPI • UART • MQTT)

Project Report

Student Name	_____
Register Number	_____
Department / Class	_____
Institution	_____
Guide / Faculty	_____
Academic Year	2026–2027

A practical learning resource combining protocol explanations, embedded code examples, simulation plans, verification checklists and AI-assisted study material.

Prepared: 30 October 2026

Project use and verification note

How to use this report: Replace the cover placeholders, add your own simulator screenshots, and record the actual results from your chosen simulator before submission. The code and test procedures are provided as reproducible examples; this document does not claim that a Wokwi/Tinkercad simulation was executed. A test is only “verified” after you run it and record evidence.

This distinction is important for academic honesty. The expected observations below are predictions based on the protocol design, not fabricated test results.

Abstract

This project develops an AI-assisted learning resource for four widely used communication protocols: UART, I2C, SPI and MQTT. UART, I2C and SPI are commonly used for communication between embedded devices and peripherals. MQTT is a lightweight publish/subscribe messaging protocol used by networked and Internet of Things (IoT) applications. The project explains each protocol at beginner and advanced levels, provides representative Arduino/ESP32 code, compares their characteristics, and defines a simulation-based verification plan. It also demonstrates how AI can help generate explanations, code drafts, quizzes and presentations while requiring human checking against authoritative specifications and actual simulator output.

Keywords: Embedded systems, IoT, UART, I2C, SPI, MQTT, AI tutor, simulation, verification.

Table of contents

• 1. Introduction and objectives
• 2. Protocol foundations
• 3. UART
• 4. I2C
• 5. SPI
• 6. MQTT
• 7. Comparison of protocols
• 8. Simulation and verification plan
• 9. Error analysis
• 10. AI tutor workflow and prompt library
• 11. Glossary
• 12. Twelve-question quiz and answer key
• 13. Seven-slide presentation outline
• 14. Automation workflow
• 15. Limitations and responsible AI use
• 16. Conclusion
• 17. References and submission checklist

1. Introduction and objectives

Embedded systems contain processors, sensors, displays, memory devices and network interfaces. These components must exchange information reliably. A communication protocol defines the rules for that exchange: signal lines, timing, addressing, data format, error handling and device roles.

Project question

How can AI explain embedded and IoT protocols clearly, and can its examples be verified in simulation?

Objectives

- Explain UART, I2C, SPI and MQTT at beginner and advanced levels.
- Use reliable technical references to check definitions, signal roles, timing and protocol features.
- Generate short example programs for each protocol and define how to test them.
- Identify common wiring, addressing, configuration and explanation mistakes.
- Prepare a comparison table, glossary, quiz, diagrams and presentation outline.
- Use AI as a learning assistant while keeping technical verification under human control.

Scope and assumptions

The code examples use Arduino-style C++ on an Arduino Uno for UART, I2C and SPI, and an ESP32 using the Arduino core plus the PubSubClient library for MQTT. Pin assignments are board-specific. Always check the exact board pinout and voltage levels before wiring. MQTT requires a network connection and a broker; it is not a direct pin-to-pin bus.

Required AI skills addressed

- Prompting and iterative refinement
- Beginner/advanced explanation generation
- Code generation and code review
- Simulation planning and result interpretation
- Quiz and answer-key design
- Presentation and diagram design

Tools proposed

AI and productivity tools: ChatGPT, Gemini, Perplexity, NotebookLM, Canva and/or Gamma. Simulation and verification: Wokwi, Arduino IDE/serial monitor, and a logic-analyzer view where supported. These are suggested tools; list only the tools you personally used in the final viva or tool log.

2. Protocol foundations

A protocol can be understood through five questions: (1) Who starts communication? (2) Which wires or network are used? (3) How is the receiver selected? (4) How are bits timed and grouped? (5) How does the sender know whether data was received?

Protocol	Communication type	Typical physical/network layer	Main use
UART	Asynchronous serial	TX, RX and common ground	Console, GPS, modules, MCU-to-MCU
I2C	Synchronous shared bus	SDA and SCL	Sensors, RTCs, EEPROMs
SPI	Synchronous serial	SCLK, MOSI, MISO, CS/SS	Displays, flash, fast peripherals
MQTT	Message-oriented publish/subscribe	Usually TCP/IP over Wi-Fi/Ethernet/cellular	IoT telemetry and commands

Important distinction

UART, I2C and SPI describe communication interfaces commonly implemented by microcontroller peripherals. MQTT is an application-layer messaging protocol. Comparing them is useful for choosing a system architecture, but they do not operate at the same layer.

Basic communication diagram

Embedded device A	Communication link	Embedded device B
MCU / controller	UART: TX ↔ RX	MCU / module
MCU / controller	I2C: SDA + SCL	Sensor / memory
MCU / controller	SPI: SCLK, MOSI, MISO, CS	Display / flash
IoT node	Wi-Fi / TCP/IP / MQTT broker	Dashboard / subscriber

Selecting a protocol

- Choose UART for a simple point-to-point serial link when both ends can agree on baud rate and frame format.
- Choose I2C when multiple low/medium-speed peripherals can share two signal wires and have unique addresses.
- Choose SPI when higher throughput and simple synchronous transfers are needed and extra chip-select wiring is acceptable.
- Choose MQTT when devices need to exchange messages through a network broker, including one-to-many distribution.

3. UART – Universal Asynchronous Receiver/Transmitter

Beginner explanation

UART sends data one bit at a time using a transmit wire (TX) and a receive wire (RX). It is asynchronous because there is no separate clock wire. Both devices must use matching settings, such as baud rate, number of data bits, parity and stop bits. TX on one device connects to RX on the other, and the devices share ground.

Advanced explanation

A UART frame normally contains a start bit, a configured number of data bits (commonly 8), an optional parity bit and one or more stop bits. The line is typically idle-high in common UART implementations. The receiver detects the start bit and samples the data according to the configured bit period. UART is usually full-duplex when separate TX and RX lines are used. Baud rate is the signaling rate; for ordinary binary UART framing it corresponds to the bit rate.

Typical frame (8-N-1)

Idle HIGH → Start bit (LOW) → 8 data bits (least-significant bit first in common UARTs) → No parity → Stop bit (HIGH) → Idle

Example: Arduino Uno serial monitor echo

Open the Arduino IDE Serial Monitor at 9600 baud. Type a line and the board sends it back. On an Uno, the hardware Serial pins are also connected to the USB-to-serial interface; do not connect an external device to pins 0/1 while relying on USB serial unless you understand the electrical interaction.

Arduino Uno UART echo code

```
void setup() {
  Serial.begin(9600);          // 9600 baud, default 8-N-1
  Serial.println("UART ready");
}

void loop() {
  if (Serial.available() > 0) {
    String message = Serial.readStringUntil('\n');
    Serial.print("Received: ");
    Serial.println(message);
  }
}
```

Wiring and test

- USB cable connects the Uno to the computer; open Serial Monitor and set 9600 baud.
- Enter “hello” and press Send. Expected output: “Received: hello”.
- For two external UART devices: connect device A TX to device B RX, A RX to B TX, and GND to GND.
- Check voltage compatibility. Many classic Uno boards use 5 V logic; many ESP32 boards use 3.3 V logic.

Common mistakes

- Connecting TX-to-TX instead of TX-to-RX.
- Using different baud rates or frame settings at the two ends.
- Forgetting common ground on a non-isolated UART link.
- Assuming UART includes addressing or built-in message delivery guarantees.

4. I2C – Inter-Integrated Circuit

Beginner explanation

I2C allows a controller to communicate with one or more target devices using two shared signal lines: SDA for data and SCL for clock. Each target normally responds to an address. This makes I2C convenient for connecting several sensors or memory chips without using a separate data wire for every device.

Advanced explanation

I2C uses open-drain/open-collector signaling with pull-up resistors. Devices pull a line low; they do not actively drive it high. A transfer begins with START, followed by an address and read/write bit, then data bytes. Each byte is followed by an ACK/NACK bit. A STOP condition ends the transfer. Repeated START can begin another phase without releasing the bus. Standard-mode is 100 kbit/s and Fast-mode is 400 kbit/s; higher modes are defined in the NXP specification. Actual limits depend on bus capacitance, pull-ups and device capability.

I2C transaction sketch

START → 7-bit target address + R/W → ACK → data byte → ACK → … → STOP

Example A: I2C scanner (Arduino Uno)

The scanner checks 7-bit addresses and prints those that acknowledge. It is a useful first test when a sensor is not detected.

Arduino Uno I2C scanner

```
#include <Wire.h>

void setup() {
  Wire.begin();
  Serial.begin(9600);
  while (!Serial) { }
  Serial.println("I2C scan");
}

void loop() {
  byte found = 0;
  for (byte address = 1; address < 127; address++) {
    Wire.beginTransmission(address);
    byte error = Wire.endTransmission();
    if (error == 0) {
      Serial.print("Found device at 0x");
      if (address < 16) Serial.print("0");
      Serial.println(address, HEX);
      found++;
    }
  }
  Serial.print("Devices found: ");
  Serial.println(found);
  delay(3000);
}
```

Example B: write a byte to an I2C target

This pair demonstrates a simple controller-to-target transfer. Upload the target sketch to a second compatible Arduino and the controller sketch to the first. On an Uno, SDA is A4 and SCL is A5. Both boards need a common ground and appropriate pull-ups; many modules already include pull-up resistors.

Controller sketch

```
#include <Wire.h>
const byte TARGET = 0x08;       // 7-bit address

void setup() {
  Wire.begin();                 // controller
  Serial.begin(9600);
}

void loop() {
  Wire.beginTransmission(TARGET);
  Wire.write((byte)42);
  byte error = Wire.endTransmission();
  Serial.println(error == 0 ? "Write ACK" : "Write failed");
  delay(1000);
}
```

Target sketch

```
#include <Wire.h>
volatile byte receivedValue = 0;
volatile bool newData = false;

void receiveEvent(int count) {
  while (Wire.available()) {
    receivedValue = Wire.read();
    newData = true;
  }
}

void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
  Wire.begin(0x08);             // target address
  Wire.onReceive(receiveEvent);
  Serial.begin(9600);
}

void loop() {
  if (newData) {
    noInterrupts();
    byte value = receivedValue;
    newData = false;
    interrupts();

    digitalWrite(LED_BUILTIN, value == 42 ? HIGH : LOW);
    Serial.println(value);
  }
}
```

Expected result: the controller reports “Write ACK”; the target receives 42, prints it, and turns on its built-in LED. The exact behavior of callbacks and multi-board simulation depends on the selected simulator and board model.

Common mistakes

- Using an 8-bit address notation when the library expects a 7-bit address.
- Forgetting pull-up resistors or using unsuitable pull-up values.
- Connecting SDA to SCL or selecting the wrong board pins.
- Using duplicate target addresses on the same bus without an address-selection method.
- Assuming every I2C device supports the same voltage, speed or register map.

5. SPI – Serial Peripheral Interface

Beginner explanation

SPI is a synchronous serial interface. A controller provides the clock and selects a target using a chip-select (CS/SS) line. The common signals are SCLK (clock), MOSI (controller out, target in), MISO (controller in, target out) and CS/SS. Multiple targets can share clock and data lines, but usually each target needs its own chip-select line.

Advanced explanation

SPI is commonly full-duplex: a bit can be shifted in and out during the same clock pulses. SPI does not define a universal device-address field or a single mandatory clock rate. The controller configures clock polarity (CPOL) and phase (CPHA), which together select SPI mode 0, 1, 2 or 3. The target datasheet determines supported modes, maximum clock and transfer format. Chip-select polarity is commonly active-low but must be checked for the device.

SPI signal roles

Signal	Controller role	Target role
SCLK	Outputs clock	Receives clock
MOSI	Outputs data	Receives data
MISO	Receives data	Outputs data
CS / SS	Selects target	Recognizes selection

Example: Uno SPI controller and target

The controller sends one byte (0x3C) and reads the byte returned by the target. This is a minimal demonstration; real peripherals may require command bytes, register addresses and timing delays.

Controller sketch (Arduino Uno)

```
#include <SPI.h>

const byte CS_PIN = 10;

void setup() {
  Serial.begin(9600);
  pinMode(CS_PIN, OUTPUT);
  digitalWrite(CS_PIN, HIGH);
  SPI.begin();
}

void loop() {
  SPI.beginTransaction(SPISettings(1000000, MSBFIRST, SPI_MODE0));
  digitalWrite(CS_PIN, LOW);
  byte received = SPI.transfer(0x3C);
  digitalWrite(CS_PIN, HIGH);
  SPI.endTransaction();

  Serial.print("Received: 0x");
  Serial.println(received, HEX);
  delay(1000);
}
```

Target sketch (ATmega328P / Uno)

```
#include <SPI.h>

volatile byte response = 0xA5;
volatile byte received = 0;

ISR(SPI_STC_vect) {
  received = SPDR;       // byte received from controller
  SPDR = response;       // byte shifted back next
}

void setup() {
  pinMode(MISO, OUTPUT); // target drives MISO
  pinMode(SS, INPUT);
  SPCR |= _BV(SPE);      // enable SPI in target mode
  SPI.attachInterrupt();
  Serial.begin(9600);
}

void loop() {
```

```
  Serial.print("Received: 0x");
  Serial.println(received, HEX);
  delay(1000);
}
```

Wiring on Arduino Uno: controller D13 SCLK → target D13; D11 MOSI → D11; D12 MISO → D12; D10 CS → target D10/SS; and GND → GND. On the target board, the hardware SS pin must be configured appropriately. The simple interrupt example is AVR-specific and should not be copied unchanged to other microcontrollers.

Expected result: the target is selected while CS is low; the controller clocks one byte; the received byte should be the target's response (0xA5) if the timing and target implementation are correct. Confirm with the simulator's serial output or logic analyzer.

Common mistakes

- Swapping MOSI and MISO.
- Selecting the wrong SPI mode or clock speed for the peripheral.
- Leaving CS inactive or selecting multiple targets simultaneously.
- Assuming SPI includes built-in addressing, ACK or error correction.
- Using an AVR-specific target sketch on a board with a different SPI peripheral architecture.

6. MQTT – Message Queuing Telemetry Transport

Beginner explanation

MQTT is a lightweight messaging protocol used in IoT. A device publishes a message to a topic. A broker receives it and forwards it to clients that subscribed to matching topics. The publisher and subscriber do not need to know each other's network address or be online at the same time, depending on broker and session settings.

Advanced explanation

MQTT uses a client–server model with a broker and topic-based publish/subscribe messaging. MQTT 3.1.1 defines three Quality of Service levels: QoS 0 (at most once), QoS 1 (at least once, duplicates possible) and QoS 2 (exactly once protocol delivery). MQTT 5.0 adds features such as reason codes, session/message expiry and enhanced properties. MQTT commonly runs over TCP; WebSocket transport is also used in some deployments. Security should include authentication and TLS where supported.

MQTT architecture

Publisher (sensor node)	→	Broker	→	Subscriber (dashboard)
Publishes to topic:		sensor/room1/temp		Subscribes to topic:
Payload: 25.4		Routes matching messages		Displays 25.4

Example: ESP32 publishes a demo message

Install the PubSubClient library in Arduino IDE. Replace Wi-Fi credentials for a private test network and use a broker you are authorized to access. This example uses the public HiveMQ broker for demonstration; public brokers are shared and untrusted. Do not publish personal, confidential or sensitive data. For real deployments use authenticated TLS and a controlled broker.

ESP32 MQTT publisher/subscriber demo

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";
const char* broker = "broker.hivemq.com";

WiFiClient wifiClient;
PubSubClient mqtt(wifiClient);

void connectWiFi() {
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
  }
}

void reconnectMQTT() {
  while (!mqtt.connected()) {
    String clientId = "esp32-demo-" + String((uint32_t)ESP.getEfuseMac(), HEX);
    if (mqtt.connect(clientId.c_str())) {
      mqtt.subscribe("capstone24/demo/command");
    } else {
      delay(2000);
    }
  }
}

void callback(char* topic, byte* payload, unsigned int length) {
  Serial.print("Message on ");
  Serial.print(topic);
  Serial.print(": ");
  for (unsigned int i = 0; i < length; i++) {
    Serial.print((char)payload[i]);
  }
  Serial.println();
}

void setup() {
  Serial.begin(115200);
  connectWiFi();
  mqtt.setServer(broker, 1883);
  mqtt.setCallback(callback);
}
```

```
void loop() {
  if (!mqtt.connected()) reconnectMQTT();
  mqtt.loop();

  static unsigned long lastSend = 0;
  if (millis() - lastSend > 5000) {
    lastSend = millis();
    mqtt.publish("capstone24/demo/temperature", "25.4");
  }
}
```

Test steps

- Create an ESP32 project in Wokwi or use a physical ESP32 board with Wi-Fi access.
- Install PubSubClient and enter valid Wi-Fi credentials in the local project only; do not include real credentials in a report or screenshot.
- Start a second MQTT client (or a broker dashboard) and subscribe to capstone24/demo/temperature.
- Run the ESP32 sketch. Expected behavior: a message with payload 25.4 appears about every five seconds.
- Publish a test command to capstone24/demo/command and confirm the ESP32 serial monitor prints the received topic and payload.

MQTT pitfalls

- Using a broker hostname that is unreachable from the simulator/network.
- Using the wrong port or TLS setting (1883 is commonly plain TCP; 8883 commonly TLS).
- Subscribing to a different topic spelling or case.
- Forgetting mqtt.loop(), which services the client connection and incoming packets.
- Assuming QoS 1 means no duplicates, or QoS 2 guarantees application-level processing exactly once.

7. Comparison of protocols

Feature	UART	I2C	SPI	MQTT
Type	Asynchronous serial	Synchronous bus	Synchronous serial	Network messaging
Wires / medium	TX, RX, GND	SDA, SCL (+ GND)	SCLK, MOSI, MISO, CS	IP network
Clock	No separate clock	SCL clock	SCLK clock	No shared hardware clock
Device selection	Usually point-to-point	Address field	Chip-select line	Topic + broker
Duplex	Full-duplex typical	Half-duplex data phases	Full-duplex typical	Publish/subscribe messaging
Speed	Configured baud rate	Common 100/400 kbit/s modes	Device-specific, often high	Network/broker dependent
Acknowledgement	No protocol ACK in basic UART	ACK/NACK per byte	No universal ACK	QoS delivery levels
Topology	Usually two endpoints	Shared multi-device bus	Controller with selected targets	Many clients via broker
Best fit	Simple serial links	Many low-speed peripherals	Fast peripheral transfers	IoT telemetry/commands

Decision guide

Requirement	Suggested choice	Reason
Debug messages from a microcontroller	UART	Simple serial console and low wiring complexity
Several sensors sharing two signal wires	I2C	Addressed shared bus
Fast display or external flash	SPI	Synchronous, high-throughput peripheral interface
Send sensor readings to cloud/dashboard	MQTT	Broker-based network distribution

There is no single “best” protocol. Selection depends on distance, throughput, number of devices, wiring, power, cost, network availability, security and the requirements of the connected components.

8. Simulation and verification plan

Recommended simulator: Wokwi. Its documentation describes support for Arduino and ESP32 projects, Wi-Fi simulation, MQTT/HTTP use cases and a virtual logic analyzer for observing signals such as UART, I2C and SPI. Simulator support can vary by board and component, so confirm that the selected project configuration supports the exact feature before claiming a successful run.

Test matrix

Test ID	Protocol	Setup	Pass condition / expected observation	Evidence to capture
T1	UART	Uno + Serial Monitor at 9600	Typed line is echoed without corrupted characters	Sketch + serial monitor screenshot
T2	I2C scan	Uno + I2C target/module	Scanner lists the known 7-bit address	Wiring + scan output
T3	I2C write	Two boards, target 0x08	Controller reports ACK; target receives 42	Both serial outputs
T4	SPI	Two Uno/AVR boards	Controller reads target response 0xA5	Wiring + logic analyzer / serial output
T5	MQTT	ESP32 + broker + subscriber	Subscriber receives 25.4 on correct topic	Broker/subscriber + serial output

Verification procedure

- Record simulator name, board model, library versions, date and project link.
- Build one test at a time. First verify power, ground and pin mapping; then run the program.
- Capture raw evidence: serial monitor output, logic-analyzer trace, broker message or simulator log.
- Compare observed signal order and settings against the relevant datasheet/specification.
- Repeat after deliberately correcting one known mistake, such as wrong UART baud rate or wrong I2C address.
- Record PASS only when the observed result meets the pass condition. Record FAIL or NOT TESTED otherwise.

Results log – fill after running the simulator

Test	Actual result	PASS / FAIL / NOT TESTED	Screenshot / project evidence
T1 UART	_____	_____	_____
T2 I2C scan	_____	_____	_____
T3 I2C write	_____	_____	_____
T4 SPI	_____	_____	_____
T5 MQTT	_____	_____	_____

Verification status for this report: procedures and expected results are specified, but simulator execution evidence must be added by the student. Do not change NOT TESTED to PASS until you have run the relevant test.

9. Error analysis and before/after comparison

Protocol	Example error / misleading claim	Why it is wrong	Correction
UART	TX connects to TX	Transmit output must feed the other device's receive input	Cross TX↔RX and share ground
UART	UART always uses 9600 baud	Baud rate is configurable	Match both ends to the selected rate
I2C	0xA0 is always the 7-bit address	Some datasheets show an 8-bit address byte including R/W	Use the library's expected 7-bit address; check datasheet
I2C	SDA/SCL can be driven high by every device	I2C uses open-drain/open-collector behavior	Use suitable pull-up resistors
SPI	SPI mode does not matter	CPOL/CPHA must match the target	Check device datasheet and configure mode
SPI	SPI has automatic device addressing	Basic SPI selects targets with CS/SS	Use separate CS lines or a supported mux
MQTT	QoS 1 means exactly once	QoS 1 can deliver duplicates	Design consumers to tolerate duplicates
MQTT	MQTT is a two-wire MCU bus	MQTT is a network messaging protocol	Use network transport and a broker

Before/after analysis

Before AI-assisted verification	After reference checking and testing
Definitions may be vague or mix protocol layers.	Each protocol is described at its correct layer.
Pin names and addresses may be guessed.	Pin mapping is checked against board documentation.
Code may compile but not match the wiring.	Code, board, wiring and library assumptions are documented.
A generated answer may claim success without evidence.	Only recorded simulator output is marked as verified.
Quiz answers may contain ambiguous wording.	Questions are reviewed against protocol references.

AI-generated content should be treated as a draft. A convincing explanation is not proof that the code compiles, the wiring is correct or the simulator supports a particular device.

10. AI tutor workflow and prompt library

Proposed workflow

Stage	AI task	Human verification
1. Collect	Find primary standards, datasheets and official board documentation	Check source authority, revision and applicability
2. Explain	Generate beginner and advanced explanations	Check terminology and technical claims
3. Code	Draft protocol example programs	Compile, inspect pins and library APIs
4. Simulate	Generate test steps and expected outcomes	Run simulator and capture evidence
5. Review	Identify likely errors and confusing statements	Confirm against trace, output and references
6. Teach	Create quiz, glossary and slide outline	Check answer key and clarity

Prompt library – copy and adapt

Prompt 1 – Beginner explanation

Explain [UART/I2C/SPI/MQTT] to a first-year embedded-systems student in simple language. Define every technical term and give one practical example. Do not invent specifications; list claims that need checking.

Prompt 2 – Advanced explanation

Explain [protocol] at an engineering level. Cover topology, signal lines, timing/frame structure, addressing or selection, acknowledgement, speed limits and common failure modes. Distinguish standard-defined behavior from device-specific behavior.

Prompt 3 – Code generation

Write a minimal [board] example for [protocol]. State board assumptions, pin mapping, libraries and configuration. Add comments, expected serial output and common compile/runtime issues. Do not claim it has been tested.

Prompt 4 – Verification

Create a step-by-step simulator test for this code and wiring. Give measurable pass/fail conditions and identify what evidence should be captured. Separate expected results from observed results.

Prompt 5 – Error review

Review this explanation/code for wrong pins, wrong addresses, timing errors, unsupported API calls and misleading claims. Return a table with issue, reason, correction and source to consult.

Prompt 6 – Quiz generation

Create 12 multiple-choice questions on UART, I2C, SPI and MQTT, balanced across all four. Provide one correct answer and a short explanation for each. Avoid ambiguous wording.

Prompt 7 – Presentation

Create a seven-slide student presentation covering the problem, protocol basics, four protocol explanations, comparison, simulation results and conclusion. Keep each slide concise.

AI tool log – complete honestly

Tool	Intended use	Used? / date	Output reviewed
ChatGPT	Explanation, code draft, quiz	_____	_____
Gemini	Alternative explanation / review	_____	_____
Perplexity	Reference discovery	_____	_____
NotebookLM	Study notes from selected references	_____	_____
Canva or Gamma	Slide design	_____	_____
Wokwi	Simulation and logic traces	_____	_____

11. Glossary

Term	Meaning
ACK / NACK	Acknowledge / Not Acknowledge response in protocols that define it.
Baud rate	Number of signaling symbols per second; in common binary UART each symbol carries one bit.
Broker	MQTT server that accepts publications and routes them to subscribers.
Bus	Shared communication path used by multiple devices.
CPOL	SPI clock polarity; defines the clock's idle level.
CPHA	SPI clock phase; defines which clock edge samples or shifts data.
CS / SS	Chip Select / Slave Select; selects an SPI target.
Full-duplex	Both directions can transfer data simultaneously.
MISO	Master/Controller In, Slave/Target Out.
MOSI	Master/Controller Out, Slave/Target In.
QoS	MQTT Quality of Service delivery level.
SCL	I2C serial clock line.
SDA	I2C serial data line.
START / STOP	I2C bus conditions that mark the beginning/end of a transaction.
Topic	MQTT hierarchical label used to route messages.
TX / RX	Transmit / Receive signal names in UART.

12. Twelve-question quiz

Choose the best answer for each question.

1. Which UART signal sends data out of a device?

A) RX B) TX C) SCL D) MISO

Answer: B – TX is the transmit output.

2. Why must two UART devices use matching baud rates?

A) To share an address B) To sample bits at compatible timing C) To generate I2C ACK D) To select SPI mode

Answer: B – Both receivers must sample the frame at compatible timing.

3. Which two lines form the basic I2C bus?

A) TX/RX B) MOSI/MISO C) SDA/SCL D) CS/SCLK

Answer: C – SDA carries data and SCL carries clock.

4. What normally follows an I2C address/data byte?

A) ACK/NACK bit B) Stop bit only C) MQTT topic D) SPI mode bit

Answer: A – The receiver signals acknowledge or not acknowledge.

5. What is the standard 7-bit I2C address range commonly used for general addressing?

A) 0–255 all valid B) 1–126 scanner range C) 128–255 only D) 0–1023

Answer: B – A basic scanner often checks 1 through 126; reserved addresses are excluded.

6. Which SPI signal is driven by the controller to provide timing?

A) SDA B) SCLK C) RX D) TX

Answer: B – SCLK is the serial clock.

7. What do CPOL and CPHA configure?

A) UART baud B) I2C address C) SPI clock polarity and phase D) MQTT QoS

Answer: C – They define SPI clock idle level and sampling/shift edge relationship.

8. How does basic SPI usually select one target?

A) Topic name B) I2C address C) Chip-select line D) Baud rate

Answer: C – A CS/SS line selects the target.

9. What is the role of an MQTT broker?

A) Generate UART clock B) Route messages between publishers and subscribers C) Provide I2C pull-ups D) Select SPI mode

Answer: B – The broker receives publications and forwards matching messages.

10. Which MQTT QoS can allow duplicate delivery?

A) QoS 0 B) QoS 1 C) QoS 2 D) None

Answer: B – QoS 1 is at least once, so duplicates can occur.

11. Which protocol is an application-layer IoT messaging protocol rather than a microcontroller peripheral bus?

A) UART B) I2C C) SPI D) MQTT

Answer: D – MQTT is a network messaging protocol.

12. What is the strongest evidence that a generated code example works?

A) AI says it works B) It looks short C) A repeatable test with recorded simulator/hardware output D) It has many comments

Answer: C – Reproducible test evidence is required.

13. Seven-slide presentation outline

Use this outline in PowerPoint, Canva or Gamma. Add your own screenshots from the simulator to Slide 6.

Slide 1 – Title and problem

CAPSTONE 24: AI Tutor for Communication Protocols. Project question: How can AI explain embedded/IoT protocols clearly, and how can examples be checked?

Slide 2 – Protocol overview

UART, I2C and SPI are common embedded serial interfaces. MQTT is a network publish/subscribe protocol. Show where each fits in a system.

Slide 3 – UART and I2C

UART: TX/RX, asynchronous, matching baud/frame. I2C: SDA/SCL, addressed shared bus, START/ACK/STOP.

Slide 4 – SPI and MQTT

SPI: SCLK/MOSI/MISO/CS, modes 0–3, full-duplex transfers. MQTT: publisher, broker, subscriber, topics and QoS.

Slide 5 – Comparison

Show the comparison table. Explain why the best protocol depends on wiring, speed, topology, power and network needs.

Slide 6 – Simulation and verification

Show Wokwi setup, code, serial monitor or logic-analyzer evidence. Include test IDs, actual results and PASS/FAIL status. Never show expected results as actual results.

Slide 7 – Conclusion

AI accelerates learning and drafting, but specifications, datasheets, compilation and simulation evidence establish correctness.

Suggested diagrams to insert

- UART frame: start bit, data bits, optional parity, stop bit.
- I2C transaction: START, address+R/W, ACK, data, ACK, STOP.
- SPI wiring: shared SCLK/MOSI/MISO and separate CS lines.
- MQTT architecture: publisher → broker → subscriber.

14. Automation workflow

Automation selected: generate a quiz form from a reviewed question bank. The workflow can be implemented with Google Forms plus Apps Script, or a no-code automation service such as Zapier/Make. The form should be generated only after a human has approved the question text and answer key.

Workflow steps

- Store the 12 reviewed questions, options and correct answers in a spreadsheet.
- Trigger an automation when a row is marked APPROVED.
- Create a quiz form and add each question with its answer options.
- Set the correct answer and feedback for each question.
- Save the form link in the project log and test it using a separate sample response.
- Check that scoring and answer feedback match the approved answer key.

Automation validation

Check	Expected behavior
Question count	Exactly 12 questions are created.
Options	Every question has the intended options; no option is missing.
Answer key	Correct option matches the reviewed answer key.
Scoring	A test submission receives the expected score.
Privacy	Collect only necessary information; do not publish student response data.

The automation design is specified here; actual form creation must be performed in the student's own account and tested. Do not claim an automation ran unless a working form and test response are available.

15. Limitations and responsible AI use

- AI can produce incorrect pin numbers, addresses, API names, timing values or unsupported code. Always check the exact board and peripheral datasheet.
- An explanation can sound confident while confusing protocol layers or terminology. Compare key claims with standards and official documentation.
- Simulator models may not reproduce every electrical effect, timing limit or hardware fault. Physical hardware testing may still be required.
- Do not upload Wi-Fi passwords, API keys, private broker credentials or personal information to AI tools or public code repositories.
- Use authenticated encrypted connections for real IoT deployments; a public demonstration broker is not a secure production design.
- Respect software licenses, tool terms and institutional academic-integrity rules. Disclose AI assistance where required.
- Keep a record of prompts, revisions, sources, code versions and actual test evidence.

Responsible AI statement

AI was used as a learning and drafting assistant for explanations, code structure, quiz design and presentation planning. Technical claims must be checked against the cited references, and code must be compiled and tested. AI-generated content is not treated as a substitute for original understanding, datasheets, standards or experimental evidence.

16. Conclusion

This capstone organizes four important communication technologies into one learning resource. UART provides a straightforward asynchronous serial link; I2C provides an addressed two-wire bus; SPI provides a clocked peripheral interface with separate data paths and chip selection; and MQTT provides broker-based messaging for IoT systems. AI can make these topics easier to learn by adapting explanations, drafting code and generating quizzes. However, reliable engineering depends on checking primary references, confirming board-specific wiring, compiling code and recording repeatable simulation or hardware results.

The main outcome is not simply AI-generated text. It is a repeatable learning process: explain → reference-check → implement → simulate → observe → correct → document.

17. References

Use the latest revision available from the official publisher when submitting. Access dates below reflect preparation of this report (30 October 2026).

Ref.	Reference and use
[1]	NXP Semiconductors, *UM10204: I2C-bus specification and user manual*, Rev. 7.0, 1 Oct 2021. Primary reference for I2C signaling, START/STOP, ACK/NACK, addressing and speed modes. https://www.nxp.com/docs/en/user-guide/UM10204.pdf
[2]	OASIS, *MQTT Version 3.1.1*, OASIS Standard. Primary reference for MQTT publish/subscribe, broker behavior and QoS. https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html
[3]	OASIS, *MQTT Version 5.0*, OASIS Standard. Reference for MQTT 5 features and protocol behavior. https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html
[4]	Arduino, *Language Reference*. Official reference for Serial, SPI and Wire APIs. https://docs.arduino.cc/language-reference/
[5]	Espressif, *Arduino-ESP32 Serial (UART) API*. Reference for ESP32 UART implementation and HardwareSerial. https://docs.espressif.com/projects/arduino-esp32/en/latest/api/serial.html
[6]	Wokwi, *Welcome to Wokwi*. Simulator capability overview, including supported boards, Wi-Fi simulation and virtual logic analyzer. https://docs.wokwi.com/
[7]	Arduino, *Arduino UNO R3 documentation*. Board pinout and hardware reference. https://docs.arduino.cc/hardware/uno-rev3/
[8]	Nick O'Leary, *PubSubClient* library documentation and examples. MQTT client library used in the ESP32 sketch. https://pubsubclient.knolleary.net/

Final submission checklist

- Cover page completed with student and institution details.
- Beginner and advanced explanations for all four protocols included.
- Definitions, timing, addresses and pin assignments checked against official sources.
- Code compiled and tested on the selected board/simulator; screenshots attached.
- Simulation results log completed with honest PASS/FAIL/NOT TESTED status.
- Comparison table, glossary, 12-question quiz and answer key included.
- Four protocol diagrams or clear screenshots added to the presentation.
- Seven-slide presentation prepared and rehearsed.
- One automation created and tested, with a form link or evidence attached.
- AI prompt library and tool-use log completed honestly.
- Before/after analysis, limitations and responsible-use statement included.
- Final demo rehearsed with working code and saved evidence.

— End of Project Report —